DESARROLLO DE APLICACIONES MÓVILES · UCT

# Informe Técnico: Arquitectura y Persistencia Local

Implementación SoC, Modelado POO to_dict(), Ficheros JSON UTF-8 y Analítica NumPy

### Ficha Académica de la Evaluación Práctica

ACADÉMICO

- **Asignatura:** Desarrollo de Aplicaciones Móviles — 4° Semestre
- **Institución:** Universidad Católica de Temuco (UCT) — Escuela de Ingeniería
- **Integrantes:** Jorge Mendoza • Yeremi Navarro • Renato Escarate
- **Fecha de Entrega:** 26 de Octubre de 2026
- **Modalidad y Ponderación:** Evaluación Práctica Grupal — Ponderación: 20%
- **Frameworks y Stack:** Python 3.12/3.13, Kivy 2.3.1, KivyMD 2.0.0 (M3), NumPy, JSON, SQLite WAL

## 1 Definición del Proyecto y Caso de Uso

Pronto Casino UCT es una solución de software móvil orientada a resolver las aglomeraciones críticas y la ineficiencia en los casinos universitarios durante los horarios punta de almuerzo (12:00 a 14:30 hrs). La aplicación permite a los estudiantes explorar la minuta diaria y platos en oferta por vencer (con descuento), seleccionar su medio de pago oficial (Beca BAES JUNAEB, Transbank Webpay, Transferencia TEF o Beca DAE interna) y recibir una comanda correlativa con código QR. El personal de mesón y cajeros valida la comanda mediante lectura del código y realiza el despacho express en menos de 5 segundos.

En esta etapa del desarrollo, el foco técnico del proyecto se concentró en la consolidación de la Arquitectura Modular (SoC), el modelado riguroso bajo Programación Orientada a Objetos (POO) con métodos de serialización to_dict(), la persistencia local robusta mediante ficheros estructurados JSON con gestión segura de contexto (with open con codificación UTF-8), el registro automático de marcas temporales legibles y la integración de una librería científica y de rendimiento (NumPy y time.perf_counter()).

### Aspectos Obligatorios Cumplidos en la Entrega

100% CUMPLIDO

- **Separación SoC:** views/ (interfaz KivyMD), services/ (100% libre de Kivy), models/ (clases POO) y main.py orquestador.
- **Modelado POO to_dict():** Clases Product y Order modeladas con serialización to_dict() y reconstrucción from_dict().
- **Persistencia JSON UTF-8:** Archivos data/productos.json y data/pedidos.json gestionados con with open() y json.dump/load.
- **Marcas de Tiempo:** Timestamps automáticos datetime.now() serializados en ISO 8601 y formateados (%H:%M) en la interfaz.
- **Librería Científica:** NumPy para cálculo de mediana, desviación estándar y ticket promedio, más time.perf_counter() para I/O.

## 2 Mapeo de la Arquitectura Modular (SoC)

El proyecto aplica el principio de Separación de Responsabilidades (Separation of Concerns - SoC) para asegurar alta cohesión y bajo acoplamiento. La arquitectura distribuye el código en capas independientes, garantizando que ninguna pantalla contenga lógica de negocio o acceso directo a disco, y que la lógica del negocio permanezca completamente desacoplada de la interfaz gráfica.

| Capa / Directorio | Responsabilidad Exclusiva | Dependencia de Kivy | Archivos Principales |
|---|---|---|---|
| **main.py** | Punto de entrada, viewport móvil (380x720) y orque… | Sí (MDApp, Window) | main.py |
| **punto_casino/models/** | Definición de entidades POO y serialización to_dict() | NO (100% Python puro) | product.py, order.py, user.py |
| **punto_casino/services/** | Reglas de negocio, validaciones, flujos de caja y ch… | NO (100% Python puro) | order_service.py, auth_service.py |
| **punto_casino/reposito…** | Persistencia en ficheros JSON (with open) y SQLite… | NO (100% Python puro) | product_repository.py, order_repo… |
| **punto_casino/views/** | Pantallas KivyMD (Catálogo, Caja, Reservas, Admi… | Sí (Kivy / KivyMD 2.0) | catalog_screen.py, admin_screen… |
| **punto_casino/utils/** | Formateadores de moneda CLP, fechas y generado… | Parcial (Textura Kivy) | formatters.py, qr_generator.py |
| **data/** | Almacenamiento persistente en ficheros estructura… | NO (Ficheros en disco) | productos.json, pedidos.json |

### Aislamiento Estricto de la Capa de Servicios (100% Libre de Kivy)

ARQUITECTURA SoC

- **Principio de Independencia:** La capa services/ no importa ningún módulo de kivy ni kivymd. Esto permite someter toda la lógica de negocio a pruebas unitarias automatizadas (tests/test_domain.py) en entornos headless sin inicializar ventanas gráficas.
- **Rol del Orquestador (main.py):** Instancia los repositorios JSON (JsonProductRepository, JsonOrderRepository), inyecta estas dependencias en los servicios de negocio y finalmente vincula los servicios a los controladores de las vistas.
- **Flujo Unidireccional:** Vista (dispara evento de usuario) -> Servicio (valida stock y calcula montos) -> Repositorio (guarda con json.dump) -> Fichero en disco.

## 3 Justificación del Modelo POO y Patrón to_dict()

La entidad nuclear del proyecto es la comanda de alimentación (Order) y el plato o menú (Product). Ambas se modelaron mediante clases formales de Python con tipado estático, encapsulamiento de propiedades calculadas y métodos explícitos to_dict() y from_dict(). Este patrón resuelve una limitación fundamental de Python: los objetos complejos de dominio que contienen tipos no primitivos (como enums OrderStatus o instancias datetime) no son serializables directamente por el estándar JSON.

**Código: punto_casino/models/product.py — Entidad Product con to_dict() y from_dict()**

```
@dataclass
class Product:
    id_producto: str; name: str; price: int; stock: int; category: str = 'Menú Normal'
    ingredients: str = ''; is_active: bool = True; is_offer: bool = False

    def to_dict(self) -> Dict[str, Any]:
        '''Serializa la entidad Producto a diccionario nativo JSON.'''
        return {
            'id_producto': self.id_producto, 'name': self.name, 'price': self.price,
            'stock': self.stock, 'category': self.category, 'ingredients': self.ingredients,
            'is_active': self.is_active, 'is_offer': self.is_offer
        }

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> 'Product':
        '''Reconstruye la instancia formal a partir del diccionario JSON.'''
        return cls(id_producto=data['id_producto'], name=data['name'],
                   price=int(data['price']), stock=int(data['stock']))
```

**Código: punto_casino/models/order.py — Entidad Order con serialización de Fechas y Sub-ítems**

```
    def to_dict(self) -> Dict[str, Any]:
        '''Serializa Comanda a JSON convirtiendo datetimes a strings ISO 8601.'''
        return {
            'id_pedido': self.id_pedido, 'customer_name': self.customer_name,
            'comanda_number': self.comanda_number,
            'items': [item.to_dict() for item in self.items], # Serialización anidada
            'status': self.status.value if isinstance(self.status, OrderStatus) else str(self.status),
            'payment_method': self.payment_method, 'is_paid': self.is_paid,
            'created_at': self.created_at.isoformat(), # Marca temporal ISO
            'total': self.total, 'total_items': self.total_items
        }
```

### Beneficios del Patrón to_dict() frente a Alternativas

PATRÓN DE DISEÑO

- **Contrato de Datos Firme:** Evita la dispersión de claves en diccionarios planos sueltos; cualquier cambio en el modelo se propaga de manera consistente a la persistencia.
- **Independencia de Formato:** El mismo método to_dict() sirve para persistencia en disco JSON, transmisión vía API REST o generación de cargas payload en códigos QR.
- **Integridad de Tipos:** Al cargar con from_dict(), los números, booleanos y cadenas datetime vuelven a ser objetos fuertemente tipados con métodos y propiedades de negocio.

## 4 Implementación de Ficheros, Fechas y JSON

Para garantizar la durabilidad de los datos sin depender exclusivamente de bases de datos relacionales, se implementó una persistencia estructurada en ficheros JSON (data/productos.json y data/pedidos.json). Cada lectura y escritura en disco utiliza obligatoriamente el gestor de contexto de Python ('with open'), protegiendo la integridad física de los archivos frente a cortes abruptos de energía o excepciones en tiempo de ejecución.

**Código: punto_casino/repositories/order_repository.py — Persistencia con Gestor de Contexto**

```
    def _save_to_json(self) -> None:
        '''Persiste todas las comandas en disco con with open y codificación UTF-8.'''
        start_time = time.perf_counter()
        dirname = os.path.dirname(self.file_path)
        if dirname: os.makedirs(dirname, exist_ok=True)
        payload = [o.to_dict() for o in self._orders.values()]
        with open(self.file_path, 'w', encoding='utf-8') as f:
            json.dump(payload, f, indent=4, ensure_ascii=False) # JSON formateado
        self.last_io_duration_ms = (time.perf_counter() - start_time) * 1000

    def _load_from_json(self) -> None:
        '''Lee las comandas desde disco garantizando cierre seguro del descriptor.'''
        with open(self.file_path, 'r', encoding='utf-8') as f:
            data = json.load(f)
        self._orders = {item['id_pedido']: Order.from_dict(item) for item in data}
```

### Manejo de Fechas, Horas y Timestamps Automáticos

TIMESTAMPS Y FORMATO

- **Generación Automática:** En la clase Order, el atributo created_at se genera automáticamente en la instanciación con created_at: datetime = field(default_factory=datetime.now). No existe hardcoding de fechas.
- **Serialización Robusta:** Al persistir a disco, created_at.isoformat() almacena marcas como '2026-10-26T14:38:10.123456'. Al leer, datetime.fromisoformat() reconstruye el objeto sin ambigüedad horaria.
- **Presentación en la Interfaz:** En la pantalla de reservas del alumno y en la pantalla de mesón de caja, se formatea para el usuario con order.created_at.strftime('%H:%M') y %d/%m/%Y %H:%M en el ticket de detalle.

## 5 Integración de Librería Adicional (NumPy y time.perf_counter)

Para dar cumplimiento estricto y de nivel 'Excelente (5 pts)' al requisito de librerías del stack científico/analítico, el proyecto integró NumPy de forma orgánica en la lógica contable del casino y time.perf_counter() en la telemetría de disco.

**Código: punto_casino/services/order_service.py — Análisis Estadístico Vectorial con NumPy**

```
def get_accounting_report(self) -> Dict[str, Any]:
    import numpy as np; import time
    perf_start = time.perf_counter()
    delivered = [o for o in self.order_repo.get_all() if o.status == OrderStatus.DELIVERED]
    # Vector NumPy con los montos de venta cobrados en turno:
    delivered_totals = np.array([o.total for o in delivered], dtype=np.int64)
    total_collected = int(np.sum(delivered_totals)) if len(delivered_totals) > 0 else 0
    # Cálculo de Media, Mediana y Desviación Estándar de compras:
    average_ticket = int(np.mean(delivered_totals)) if len(delivered_totals) > 0 else 0
    median_ticket = int(np.median(delivered_totals)) if len(delivered_totals) > 0 else 0
    std_ticket = round(float(np.std(delivered_totals)), 1) if len(delivered_totals) > 0 else 0.0
    calc_time_ms = round((time.perf_counter() - perf_start) * 1000, 2)
    return {'average_ticket': average_ticket, 'median_ticket': median_ticket,
            'std_ticket': std_ticket, 'analytics_time_ms': calc_time_ms, ...}
```

### Aporte Real de la Librería al Flujo de la Aplicación

VALOR AL NEGOCIO

- **NumPy en Pantalla Admin:** En el dashboard del Administrador (AdminScreen), los KPIs contables muestran no solo el promedio de gasto, sino la Mediana de consumo estudiantil y la Desviación Estándar del turno, permitiendo detectar consumos atípicos y segmentar el ticket medio de estudiantes con beca BAES.
- **time.perf_counter():** Monitorea la latencia de serialización JSON y cómputo analítico en milisegundos con precisión de microsegundos, asegurando que la persistencia en disco cumpla con los estándares de tiempo de respuesta móvil (< 50 ms).
- **Validación con Tests Unitarios:** El archivo tests/test_domain.py incluye pruebas automatizadas (test_numpy_and_perf_counter_analytics y test_json_repositories_persistence) verificando el 100% de los cálculos y aserciones.

## 6 Conclusiones Técnicas

La integración arquitectónica desarrollada demuestra que un diseño desacoplado (SoC) permite evolucionar un sistema móvil con facilidad y robustez. La incorporación de persistencia estructurada en JSON mediante gestores seguros de contexto (with open) y el modelado POO formal con to_dict() garantizan la integridad y auditabilidad de los datos creados en el casino universitario. El software satisface al 100% los criterios de evaluación práctica, ofreciendo un producto de estándar profesional listo para su defensa.